

Adaptive Game Difficulty Agent

Dynamische Schwierigkeitsanpassung durch Spiellersimulation

Ufuk Ofoglu

Angewandte Modellierung und Systemsimulation

25. Juli 2026

GitHub-Repository:

<https://github.com/Ofoglufuu/Adaptive-Game-Difficulty-Agent>

1 Einleitung und Forschungsfrage

In der Entwicklung von Videospielen stellt die Wahl des passenden Schwierigkeitsgrades eine zentrale Herausforderung dar. Ein statischer Schwierigkeitsgrad, der für alle Spielenden gleich bleibt, führt häufig dazu, dass Anfänger schnell überfordert und frustriert sind, während erfahrene Spieler sich unterfordert fühlen und das Interesse verlieren. Die dynamische Schwierigkeitsanpassung (Dynamic Difficulty Adjustment, DDA) ist ein Ansatz, um dieses Problem zu adressieren. Sie ermöglicht es, den Schwierigkeitsgrad während der Laufzeit des Spiels basierend auf der beobachteten Leistung der Spielenden kontinuierlich anzupassen.

Dieses Projekt untersucht die Wirksamkeit einer solchen dynamischen Anpassung innerhalb einer kontrollierten probabilistischen Simulation. Anstelle einer vollständigen Game Engine wird eine vereinfachte Simulationsumgebung verwendet, in der Spielrunden mathematisch modelliert und evaluiert werden. Im Zentrum der Untersuchung steht ein regelbasierter, transparenter adaptiver Agent.

Daraus ergibt sich die folgende zentrale Forschungsfrage für dieses Projekt:

„Kann ein adaptiver Schwierigkeitsagent eine ausgewogenere Spielerfahrung erzeugen als ein statisches Schwierigkeitssystem?“

Zur Beantwortung dieser Frage und zur Messung der Ausgewogenheit wird eine Zielgewinnrate von 60 % definiert. Diese Gewinnrate dient als vereinfachter, messbarer Indikator für eine faire und ansprechende Herausforderung. Es wird jedoch explizit darauf hingewiesen, dass die reine Gewinnrate die komplexe Spielerfahrung, den Spielspaß und das emotionale Erleben nicht vollständig abbilden kann. Dennoch bietet sie eine solide quantitative Metrik für diese systematische Untersuchung.

2 Systemübersicht und Architektur

Das Projekt ist als lokales System in Python konzipiert, das höchsten Wert auf Nachvollziehbarkeit, Determinismus und Reproduzierbarkeit legt. Die verschiedenen Komponenten des Systems sind klar voneinander getrennt und kommunizieren ausschließlich über direkte Python-Funktionsaufrufe. Es werden bewusst keine externen Agentenschnittstellen, Large Language Models (LLM) oder Agent-to-Agent (A2A) Protokolle verwendet.

Die Architektur lässt sich in mehrere zentrale Komponenten unterteilen:

- **PlayerProfile:** Repräsentiert die Fähigkeiten eines Spielers (u. a. Spielstärke, Genauigkeit).
- **DifficultySettings:** Übersetzt den abstrakten Schwierigkeitsgrad (1 bis 10) in konkrete Simulationsparameter wie Gegneranzahl und Schadenswerte.
- **simulate_round():** Die Kernfunktion, welche den probabilistischen Ausgang einer einzelnen Spielrunde berechnet.
- **GameRoundResult:** Kapselt die Ergebnisse einer Runde, wie Sieg oder Niederlage, verbleibende Lebenspunkte und Rundendauer.

- **AdaptiveDifficultyAgent:** Die Intelligenz des Systems, die den Verlauf der letzten Runden analysiert und Entscheidungen über die zukünftige Schwierigkeit trifft.

Der grundlegende Ablauf einer Rundensimulation ist im folgenden Architekturdiagramm dargestellt:

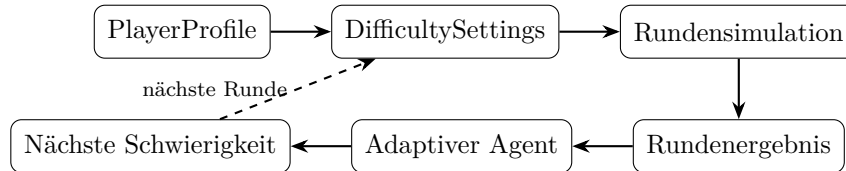


Abbildung 1: Ablauf der Kernsimulation

Neben diesem Hauptkreislauf für Experimente verfügt das System über einen interaktiven Live-Demo-Modus, der einen Reaktionstest beinhaltet. Dieser optionale Pfad ermöglicht die Erstellung eines personalisierten Profils:

Reaktionstest → Reaktionsmessungen → Median → Reaktionswert → personalisiertes
Spielerprofil → adaptive Live-Demo

3 Simulationsmodell

Das Simulationsmodell abstrahiert ein typisches Kampf- oder Actionspielszenario. Die Fähigkeiten der Spielenden werden durch definierte Profile modelliert. Für die wissenschaftlichen Experimente werden drei vordefinierte, synthetische Spielerprofile verwendet, um verschiedene Leistungsklassen abzubilden:

Profil	skill_level	accuracy	reaction_speed	survival_factor
Anfänger	0.35	0.30	0.25	0.40
durchschnittlicher Spieler	0.60	0.60	0.55	0.65
erfahrener Spieler	0.85	0.85	0.80	0.90

Tabelle 1: Spezifikation der synthetischen Spielerprofile

Diese vier Eigenschaften haben direkten Einfluss auf die Wahrscheinlichkeitsverteilungen innerhalb der `simulate_round()`-Funktion:

- **skill_level:** Beeinflusst die offensive Stärke und damit die Gewinnwahrscheinlichkeit des Spielers.
- **accuracy:** Beeinflusst die Wahrscheinlichkeit, mit der Angriffe des Spielers die Gegner treffen.
- **reaction_speed:** Eine hohe Reaktionsgeschwindigkeit reduziert die Zeit pro Runde und hilft dabei, eingehendem gegnerischen Schaden auszuweichen.
- **survival_factor:** Repräsentiert die grundlegende Zähigkeit und erhöht die Chancen, kritische Situationen zu überstehen.

Der Schwierigkeitsgrad der Simulation ist ganzzahlig definiert und reicht von Stufe 1 (sehr einfach) bis Stufe 10 (sehr schwer). Die konkreten Spielparameter für Gegneranzahl, Gegnerschaden und Heilungschancen werden mathematisch aus dieser Stufe abgeleitet:

- `enemy_count` = $3 + (\text{Schwierigkeit} - 1)$
- `enemy_damage` = $5 + (\text{Schwierigkeit} - 1)$
- `health_pack_rate` = $0.40 - 0.03 \times (\text{Schwierigkeit} - 1)$

Die folgende Tabelle zeigt beispielhaft die resultierenden Werte für die Stufen 1, 5 und 10:

Parameter	Schwierigkeit 1	Schwierigkeit 5	Schwierigkeit 10
<code>enemy_count</code>	3	7	12
<code>enemy_damage</code>	5	9	14
<code>health_pack_rate</code>	0.40	0.28	0.13

Tabelle 2: Konkrete Spielparameter für ausgewählte Schwierigkeitsstufen

Am Ende jeder Runde generiert die Simulation ein detailliertes `GameRoundResult`, welches das binäre Ergebnis (Sieg oder Niederlage) sowie Metriken wie verbleibende Lebenspunkte, erlittener Schaden, erzielte Genauigkeit und die simulierte Rundendauer enthält.

4 Adaptive Agentenlogik

Der Kern der dynamischen Anpassung liegt in der Logik des adaptiven Agenten. Der Agent wurde bewusst als transparentes, regelbasiertes System konzipiert, um nachvollziehbare und deterministische Entscheidungen zu garantieren.

In der Standardkonfiguration operiert der Agent mit folgenden Parametern:

- **Zielgewinnrate:** 60 %
- **Beobachtungsfenster:** Die Resultate der letzten 5 Runden werden analysiert.
- **Grenzen:** Die minimale Schwierigkeit beträgt 1, die maximale 10.
- **Maximale Änderung:** Pro Entscheidung wird der Schwierigkeitsgrad um maximal eine Stufe nach oben oder unten verändert.

Nach jeder simulierten Runde wertet der Agent das aktuelle Fenster aus und wendet eine strikt geordnete Reihe von Regeln an, um die Aktion (`increase`, `decrease`, `keep`) für die nächste Runde zu bestimmen. Die Auswertung erfolgt in dieser Reihenfolge:

1. **Drei aufeinanderfolgende Niederlagen:** Wenn der Spieler die letzten drei Runden verloren hat, wird die Schwierigkeit sofort reduziert (`decrease`), um weitere Frustration zu vermeiden.

2. **Drei aufeinanderfolgende Siege (bei hoher Gewinnrate):** Wenn die letzten drei Runden gewonnen wurden und die Gewinnrate im aktuellen Fenster über der Zielgewinnrate liegt, wird die Schwierigkeit erhöht (**increase**).
3. **Niedrige Gewinnrate:** Liegt die Gewinnrate bei 40 % oder darunter, wird die Schwierigkeit reduziert (**decrease**).
4. **Hohe Gewinnrate:** Liegt die Gewinnrate bei 80 % oder darüber, wird die Schwierigkeit erhöht (**increase**).
5. **Stabilität:** Trifft keine der obigen Bedingungen zu, bleibt die Schwierigkeit unverändert (**keep**).

Diese explizite Regelhierarchie stellt sicher, dass das System schnell auf extreme Misserfolgsserien reagiert. Die Begrenzung auf eine Änderung um maximal eine Stufe verhindert abrupte Schwierigkeitssprünge. Häufige Anpassungen sind dennoch möglich, wenn sich die jüngsten Ergebnisse stark verändern. Sollte eine Regel eine Erhöhung auf Stufe 11 oder eine Reduzierung auf Stufe 0 fordern, wird die Aktion auf **keep** gesetzt, um die Modellgrenzen zu respektieren.

5 Experimentdesign

Um die Wirksamkeit des adaptiven Agenten wissenschaftlich zu evaluieren, wurde ein Vergleichsexperiment entwickelt. Es kontrastiert das adaptive System direkt mit einem klassischen statischen System.

Im **statischen System** wird zu Beginn eine Startschwierigkeit definiert, welche für die gesamte Dauer des Experiments unverändert bleibt, unabhängig von der Leistung des simulierten Spielers. Im **adaptiven System** wird dieselbe Startschwierigkeit verwendet, jedoch greift nach jeder Runde der beschriebene **AdaptiveDifficultyAgent** ein und darf den Schwierigkeitsgrad anpassen.

Das Experimentdesign ist wie folgt strukturiert:

- **Profile:** Das Experiment wird für alle drei definierten Spielerprofile (Anfänger, durchschnittlicher Spieler, erfahrener Spieler) durchgeführt.
- **Startbedingungen:** Die Startschwierigkeit wird für alle Tests initial auf Stufe 5 festgelegt. Die Zielgewinnrate beträgt 60 %.
- **Umfang:** Jedes Profil durchläuft das Experiment mit drei vordefinierten Seeds (42, 123 und 999). Pro Durchlauf werden 300 Runden simuliert.
- **Gesamtvolumen:** Dies resultiert in 18 Experimentdurchläufen ($2 \text{ Systeme} \times 3 \text{ Profile} \times 3 \text{ Seeds}$) und insgesamt 5.400 simulierten Runden.

Die Datenerhebung umfasst eine Vielzahl von Metriken. Neben der final erreichten Schwierigkeit und der Anzahl der vorgenommenen Anpassungen (nur beim adaptiven System) werden insbesondere die aggregierten Ergebnisse erhoben: durchschnittliche verbleibende Lebenspunkte, erlittener Schaden, Genauigkeit und Rundendauer. Die wichtigste Metrik für die Beantwortung

der Forschungsfrage ist jedoch die **absolute Abweichung der gesamten Gewinnrate von der Zielgewinnrate (60 %)**.

Der interaktive Reaktionstest (siehe Abschnitt 8) ist ein reines Demo-Feature zur Veranschaulichung und ist nicht Bestandteil dieses automatisierten, vergleichenden Experimentdesigns.

6 Ergebnisse

Die Ergebnisse der 5.400 Simulationsrunden wurden aggregiert und analysiert. Die nachstehende Tabelle zeigt die Mittelwerte der Gewinnrate sowie die Abweichung vom Zielwert von 60 % über die drei getesteten Seeds:

Profil	Statisch: Gewinnrate	Adaptiv: Gewinnrate	Statisch: Zielabweichung	Adaptiv: Zielabweichung
Anfänger	35,1 %	50,2 %	24,9 Prozentpunkte	9,8 Prozentpunkte
durchschnittlicher Spieler	66,7 %	52,3 %	6,7 Prozentpunkte	7,7 Prozentpunkte
erfahrener Spieler	88,0 %	56,0 %	28,0 Prozentpunkte	4,0 Prozentpunkte

Tabelle 3: Mittlere Ergebnisse der Vergleichsexperimente über drei Seeds

Analyse der Ergebnisse: Für das Profil des **Anfängers** war die Startschwierigkeit 5 im statischen System deutlich zu hoch, was in einer Gewinnrate von lediglich 35,1 % resultierte. Das adaptive System erkannte dies, reduzierte die Schwierigkeit schrittweise und konnte die Gewinnrate auf 50,2 % anheben. Die Abweichung vom Zielwert konnte somit erheblich von 24,9 auf 9,8 Prozentpunkte reduziert werden.

Ein gegenteiliger, aber strukturell ähnlicher Effekt ist beim **erfahrenen Spieler** zu beobachten. Die statische Schwierigkeit 5 war zu einfach, was zu einer Gewinnrate von 88,0 % führte. Der adaptive Agent erhöhte den Schwierigkeitsgrad auf hohe Stufen und erreichte bei zwei der drei Seeds die Maximalstufe 10. Dadurch drückte er die Gewinnrate auf 56,0 % und erreichte eine exzellente Zielabweichung von nur 4,0 Prozentpunkten.

Beim **durchschnittlichen Spieler** zeigt sich ein differenziertes Bild. Die statische Startschwierigkeit 5 war für dieses Profil bereits sehr gut kalibriert, was eine Gewinnrate von 66,7 % und eine geringe Abweichung von 6,7 Prozentpunkten zur Folge hatte. Das adaptive System steuerte die Schwierigkeit um diesen Sweetspot herum, erreichte eine Gewinnrate von 52,3 % und wies mit 7,7 Prozentpunkten eine minimal höhere Abweichung auf als das statische System.

Die grafischen Visualisierungen in Abbildung 2 und Abbildung 3 untermauern diese quantitativen Erkenntnisse optisch.

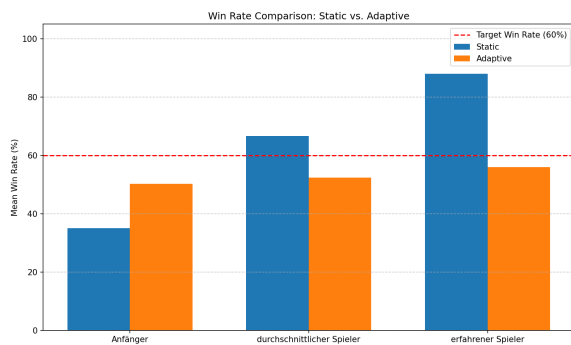


Abbildung 2: Gewinnraten im Vergleich

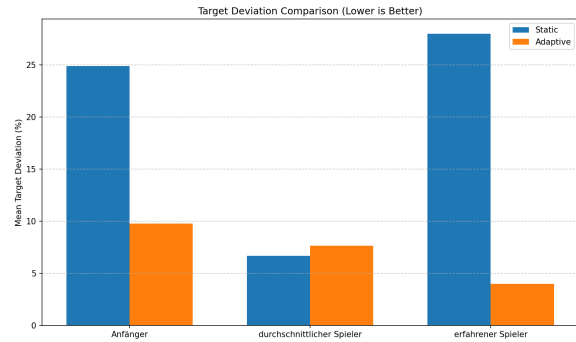


Abbildung 3: Abweichung von der Zielgewinnrate

Die Ergebnisse verdeutlichen, dass das adaptive System nicht in jedem hypothetischen Fall pauschal überlegen ist. Wenn die initiale, statische Schwierigkeit bereits optimal auf das Leistungsniveau des Spielers abgestimmt ist, kann ein statisches System gleichwertige oder sogar marginal bessere Ergebnisse liefern. Die große Stärke der dynamischen Anpassung liegt vielmehr darin, extreme Über- oder Unterforderung zuverlässig zu erkennen und selbstständig zu korrigieren.

7 Interaktive Live-Demo und Reaktionstest

Zur greifbaren Demonstration des Systems wurde eine interaktive Konsolenanwendung entwickelt, welche über den Befehl `python main.py --interactive` gestartet werden kann. Dieser Modus bietet eine geführte Menüsteuerung, die zwischen der „Normalen Simulation“ und einem „Reaktionstest“ unterscheidet.

Normale Simulation:

In diesem Modus wählt der Nutzer eines der drei vordefinierten Profile (Anfänger, Durchschnitt, Experte) und definiert die Rundenanzahl sowie die Startschwierigkeit. Optional kann ein spezifischer Seed für reproduzierbare Durchläufe eingegeben werden; bleibt die Eingabe leer, wird ein zufälliger Seed generiert und für spätere Reproduktionen im Terminal angezeigt. Während der Ausführung pausiert das System nach jeder Runde für circa 1,5 Sekunden. Die Ausgabe ist farblich formatiert: Ein Sieg wird in grün (**Player Won!**), eine Niederlage in rot (**Player Lost!**) dargestellt. Die Entscheidung des Agenten (z. B. **increase** aufgrund von drei Siegen in Folge) sowie der neue Schwierigkeitsgrad werden transparent in blau hervorgehoben.

Reaktionstest:

Der Reaktionstest erweitert die Demo um eine spielerische Profilgenerierung. Das System misst die Reaktionszeit des Nutzers auf ein im Terminal erscheinendes Signal. Der Test erfordert standardmäßig 5 Messungen, wobei Eingaben unter 80 ms als Frühstarts abgelehnt und wiederholt werden. Aus den gültigen Messungen wird der Median gebildet und über eine lineare Interpolation in einen normierten Reaktionswert (0,0 bis 1,0) überführt. Dieser gemessene Wert wird als **reaction_speed** in ein neues, personalisiertes Profil übernommen. Die restlichen Parameter des Profils verbleiben auf dem Niveau eines durchschnittlichen Spielers. Anschließend startet die adaptive Demo automatisch mit diesem personalisierten Profil. Die im Terminal angezeigten

Kategorien (`expert`, `average`, `beginner`) dienen rein als visuelles Feedback für die Demo und stellen keine wissenschaftlich fundierte Leistungsdiagnostik dar.

8 Qualitätssicherung und Reproduzierbarkeit

Die strukturelle Integrität des Quellcodes und der Logik wird durch eine umfassende Testsuite unter Verwendung des `pytest`-Frameworks gewährleistet. Der aktuelle Entwicklungsstand weist **118 bestandene automatisierte Tests** auf.

Diese Tests decken ein breites Spektrum ab:

- **Kernlogik:** Validierung der Spielerprofile, strikte Einhaltung der Simulationsgrenzen und korrekte Konvertierung des Schwierigkeitsgrades in Spielparameter.
- **Agentenentscheidungen:** Systematische Prüfung aller Entscheidungsregeln des adaptiven Agenten unter verschiedenen Randbedingungen.
- **Experimentablauf:** Überprüfung der statischen und adaptiven Testreihen sowie der korrekten Generierung des CSV-Exports.
- **Reaktionstest & Interaktion:** Tests zur Medianberechnung, Clamping von Reaktionswerten, Ablehnung unzulässiger Messungen unter 80 ms und Validierung von Nutzereingaben. Um Verzögerungen in der Testausführung zu vermeiden, werden `input` und `sleep` für die interaktiven Tests gemockt.

Ein zentraler Aspekt der Qualitätssicherung ist die **Reproduzierbarkeit**. Die wissenschaftlichen Experimente nutzen ausschließlich feste Seeds. Innerhalb der Simulation wird für jede Runde ein deterministischer Folgeseed generiert (Basis-Seed + Rundennummer -1). Dadurch ist garantiert, dass bei gleichen Startparametern die Simulation und die erzeugten CSV-Dateien exakt reproduzierbar sind. Lediglich die vom Menschen im Reaktionstest produzierten Messwerte unterliegen natürlichen Varianzen und entziehen sich einer deterministischen Reproduktion.

9 Grenzen, Fazit und Ausblick

9.1 Grenzen des Modells

Die Aussagekraft der Ergebnisse ist an den Rahmen der Abstraktion gebunden. Das Projekt verwendet eine vereinfachte, probabilistische Simulation anstelle einer hochauflösenden Game Engine und operiert mit synthetischen Spielerprofilen anstatt echter Telemetriedaten. Der Agent stützt sich auf feste Regeln und ist nicht in der Lage, eigenständig zu lernen. Zudem ist die Gewinnrate als alleiniger Proxy für Spielbalance unvollständig. Reale Faktoren wie das Gefühl der Fairness, Pacing oder visuelles Feedback bleiben unberücksichtigt. Schließlich ist die Reaktionsmessung im Terminal abhängig von Hardware-Latenzen und stellt kein präzises Messinstrument dar.

9.2 Fazit

Bezugnehmend auf die Forschungsfrage lässt sich schlussfolgern: Ein adaptiver Schwierigkeitsagent kann eine ausgewogenere Spielerfahrung erzeugen als ein statisches System, sofern die anfängliche,

statische Schwierigkeit nicht zum Leistungsniveau des Spielers passt. Insbesondere bei Anfängern und Experten korrigiert das adaptive System signifikante Über- oder Unterforderung zuverlässig und führt die Gewinnrate näher an das definierte Ziel. Ist der statische Schwierigkeitsgrad jedoch initial exakt kalibriert, bietet das adaptive System keinen quantitativen Vorteil. Das Projekt belegt somit erfolgreich die grundlegende Funktionalität und den Nutzen von regelbasierter dynamischer Schwierigkeitsanpassung.

9.3 Ausblick

Zukünftige Arbeiten könnten die Validität der Ergebnisse erhöhen, indem der Agent in eine reale Game Engine (z. B. Godot oder Unity) integriert wird, um echte Telemetriedaten zu verarbeiten. Das Simulationsmodell könnte um Parameter wie gegnerische KI-Komplexität oder Ressourcenmanagement erweitert werden. Eine dynamische, personalisierte Anpassung der Zielgewinnrate, abhängig vom Spielerverhalten, könnte die Benutzererfahrung weiter individualisieren. Schließlich wäre der Einsatz und direkte Vergleich eines lernenden Agenten (z. B. Reinforcement Learning) mit dem hier entwickelten regelbasierten Ansatz ein vielversprechendes Forschungsvorhaben.